VIT®

BHOPAL

NAME:-Omkar Kumar

REG NO:-25BAI10893

COURSE:-Fundamental of ai and ml

DATE:-20/03/2026

FACULTY:-Monica Sanket

1.PROBLEM STATEMENT

At VIT Bhopal, students residing in the hostel are required to follow a structured outing process before leaving the campus. The gate guard at the hostel entrance is responsible for verifying whether each student meets the eligibility criteria for outing or not. This process involves checking multiple conditions simultaneously — the number of outings already taken that month, whether the student has any late return penalties, and whether approvals from both the Proctor and the Hostel Warden have been obtained.

This process takes a lot of time in verifying each and every conditions manually so to make it easier and making the process smooth I have made the program in prolog to help them. They just need to fill certain options like

1. How many outings has the student taken this month?
2. How many late penalties does the student have?
3. Has the proctor approved?
4. Has the hostel approved?

And this program will instantly tell whether the student is eligible for outing or not.

2. WHY THIS PROBLEM MATTERS

The outing approval process at a university hostel is not just a formality — it directly impacts student safety, institutional accountability, and administrative efficiency. Here is why automating this process is important:

1.Student Safety: - For any educational institution like VIT-BHOPAL safety of its student is of utmost priority.

2.Consistency and Fairness:- Manual verification is inconsistent. One guard may apply rules differently than another. An automated system applies the same rules to every student every time, ensuring fairness.

3.Reducing Human Error:- A guard might tired after work and hence it can make error while this wont.

4.Speed and Efficiency:- speed and efficiency of machines like this would any day will be better then human speed and efficiency.

3.MY APPROACH OF SOLVING IT

I used prolog programming language to solve this problem . I divided this program into two files:-

01-rules.pro — Contains all the eligibility rules and the logic for determining whether the student is eligible for outing or not. This is the brain of the system.
02-main.pro — Handles user interaction. It asks the gate guard four questions and passes the answers to the rules file for evaluation.

The following rules were implemented in determining the eligibility:-
1.Outing limit:-A student can take a maximum of 2 outings per month. If they have already taken 2, they are denied.

2.Late penalty:-If a student previously returned late (after 6 PM), they receive a penalty that cancels their one outing.

3.Proctor approval:-The student must have obtained written approval from their Proctor before the outing can be considered.

4.Hostel approval:- The student must also have received approval from the hostel too. Both the approvals are mandatory.

4.KEY DECISION MADE

1.SPLITTING INTO TWO FILES:-
To make the code not messy I have decided to separate the code into two files
01-rules.pro — Contains all the eligibility rules and the logic for determining whether the student is eligible for outing or not. This is the brain of the system.
02-main.pro — Handles user interaction. It asks the gate guard four questions and passes the answers to the rules file for evaluation.

2.Using % and /* */ command
I always used these two command for comments so anyone reading the code easily understand it and able to apply it.

3.Given MIT license
I used MIT license in the repository so it should not be limited to us only. Any educational institution or anyone who want to use this system for their benefits can use this freely.

5.CHALLENGES FACED

1.BLOCK COMMENTS INSIDE RULES :-
The most time-consuming challenge was discovering that /* */ block comments cannot be used inside a Prolog rule body.

2.VARIABLE NAMING IN PROLOG
After coding in python and c++ a long time I got confused here in naming variable . There are certain rules in how we can name the variable like it should start with capital letter only.

3.UNDERSTANDING THE DOT REQUIREMENT
Here I learnt after some confusion that after every command like start. Or any input that we gave we should put a dot after that.

4.INSTALLING SWI-PROLOG ON WINDOWS
During setup, swipl command was not recognized and it took a lot of effort to finally set it up.

6.WHAT I LEARNED

1.I had coded previously in c++ and python but this project helped me learning a new language prolog. Learning new things is always fun.

2.Learning prolog also made me think about how in earlier times code used to be written in basic languages like prolog.

3. Prolog is sensitive to small syntactic details like capital letters for variables, dots at the end , and comment placement. Attention to detail is critical.

4.The importance of code structure. This project helped me to know how to separate code files to not make them messy. Separating code into different files helps us to make it look nice, avoid unnecessary bugs, and reusability of the code in future.